



方法的定义与使用

北京理工大学计算机学院
金旭亮

软件工程中的模块化原则



将一个复杂的系统划分为子模块，便于设计、实现和维护，这是设计与开发复杂软件系统的基本指导原则。



Java中的程序模块可分为“方法（函数）”、“类”、“包”和“模块”四个层次。



Java程序最基本的构造单元是类，而类中最重要的成员就是方法。



在本讲中，我们就从最基本的编程构件——方法讲起.....

什么是“方法”？

所谓“方法”，其实就是一小段完成特定数据处理工作的代码集合。它有一个名字，在需要用到这些代码的地方，只需要给出名字即可。



“方法”本质上就是“函数”，它是“函数”在面向对象领域的另一个“名字”。

Java方法的实例——JDK中的Math类

java.lang.Math类提供了通用的数学函数方法，这些数学函数方法都是**静态**方法。所以可以用“**类名.方法名**”的方式直接调用

```
/*-----下面是三角运算-----*/  
//将弧度转换角度  
System.out.println("Math.toDegrees(1.57): " + Math.toDegrees(1.57));  
//将角度转换为弧度  
System.out.println("Math.toRadians(90): " + Math.toRadians(90));  
//计算反余弦，返回的角度范围在 0.0 到 pi 之间。  
System.out.println("Math.acos(0.3): " + Math.acos(1.2));  
//计算反正弦；返回的角度范围在 -pi/2 到 pi/2 之间。  
System.out.println("Math.asin(0.8): " + Math.asin(0.8));  
//计算反正切；返回的角度范围在 -pi/2 到 pi/2 之间。  
System.out.println("Math.atan(2.3): " + Math.atan(2.3));
```

代码摘自：TestMath.java

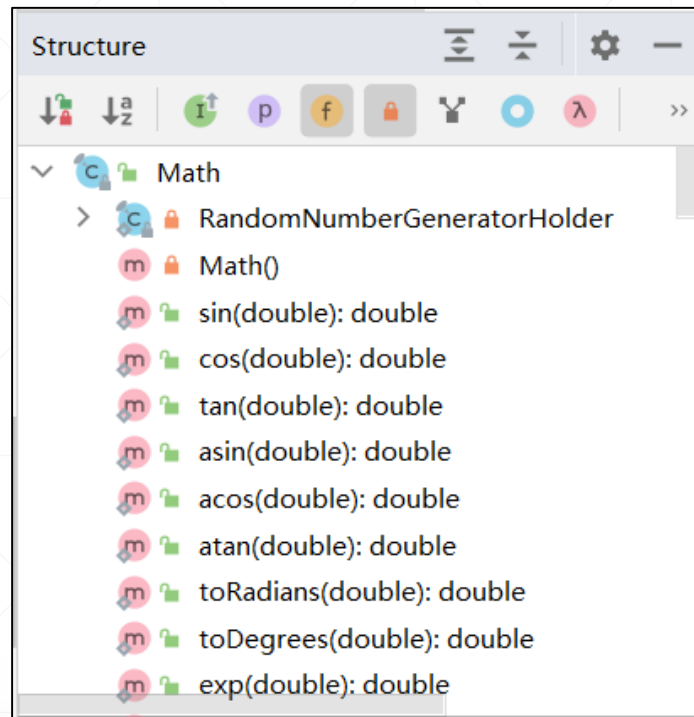


源码阅读训练

在IntelliJ中选中Math这个标识符，Ctrl + 鼠标点击，可以查看Math类的源代码：

```
1596
1597 /**
1598  * Returns the smaller of two {@code long} values. That is,
1599  * the result is the argument closer to the value of
1600  * {@link Long#MIN_VALUE}. If the arguments have the same
1601  * value, the result is that same value
1602  *
1603  * @param a an argument.
1604  * @param b another argument.
1605  * @return the smaller of {@code a} and {@code b}.
1606  */
1607 public static long min(long a, long b) {
1608     return (a <= b) ? a : b;
1609 }
```

JDK中Math类的源码



Math类大约有两千多行代码，分成很多个独立的函数，包含有相应的注释，请试着读读它的源代码。



阅读并理解他人代码，是程序员必备的基础技能，为团队协作开发所必需。

使用静态导入简化代码

从JDK5.0开始，支持一种称为“静态导入”的语法特性：

```
import static java.lang.Math.*;
```

这么干了之后，你就可以在Java代码中省略掉类名而只写静态方法名：

```
System.out.println(abs(-100));
```

上述方法完全等价于：

```
System.out.println(Math.abs(-100));
```

类方法的编写



看看JDK提供的数学函数类Math的编写方式，我们可以知道——“将方法放入类中，并将其定义为静态(static)的”是面向对象软件提供类似于C语言程序中“全局函数”的基本手段。



我们也可以开发自己的方法供其他人使用，只需创建一个类，然后为其编写声明为public的函数即可。

在Java中定义方法

- 语法格式:

```
访问权限 [static] 返回值类型 方法名( 参数列表 )  
{  
    语句...  
}
```

- 方法的返回值:

```
return 表达式;
```

上述第一行所包容的“信息”，称为此方法的“**签名**”。

这个例子中，程序员自定义了一个求平方数的静态方法square：

```
public class SquareInt {  
    public static void main(String[] args) {  
        int result;  
        for (int x = 1; x <= 10; x++) {  
            //调用自定义函数求平方数  
            result = square(x);  
            //Math库中也提供了求平方数的方法  
            //result = (int) Math.pow(x, 2);  
            System.out.println(x + "的平方数是: " + result);  
        }  
    }  
  
    // 自定义求平方数的静态方法  
    public static int square(int y) {  
        return y * y;  
    }  
}
```

方法调用

方法定义

示例：SquareInt.java

Java方法示例：随机数生成

Math.random()生成[0,1)之间的随机数



示例：RandomInt.java

这个例子说明了指定区间内随机整数的生成方法。

```
public class RandomInt {  
    public static void main(String[] args)  
    {  
        int value;  
        String output = "";  
  
        for ( int i = 1; i <= 20; i++ ) {  
            value = 1 + (int) ( Math.random() * 6 );  
            output += value + " ";  
            if ( i % 5 == 0 )  
                output += "\n";  
        }  
  
        JOptionPane.showMessageDialog( parentComponent: null,  
            output,  
            title: "生成20个从1到6的随机数",  
            JOptionPane.INFORMATION_MESSAGE );  
  
        System.exit( status: 0 );  
    }  
}
```

使用Random类生成随机数-1

JDK提供了一个Random类，可以更方便地生成随机数。

TestRandom.java

```
Random rand = new Random();  
//生成一个随机的bool数值  
System.out.println("rand.nextBoolean(): " + rand.nextBoolean());  
//创建一个包容16个元素的字节数组，使用随机数填充  
byte[] buffer = new byte[16];  
rand.nextBytes(buffer);  
System.out.println(Arrays.toString(buffer));  
//生成0.0~1.0之间的伪随机double数  
System.out.println("rand.nextDouble(): " + rand.nextDouble());
```



计算机中生成的“随机数”，是使用特定算法生成的“伪随机数”，当你重复足够多的次数时，会发现它其实还是有规律可遵循的。真的随机，要到量子物理学或大自然中去找。

使用Random类有一个问题

TestSeed.java

```
Random r1 = new Random( seed: 50);
System.out.println("第一个种子为50的Random对象");
System.out.println("r1.nextBoolean():\t" + r1.nextBoolean());
System.out.println("r1.nextInt():\t\t" + r1.nextInt());
System.out.println("r1.nextDouble():\t" + r1.nextDouble());
System.out.println("r1.nextGaussian():\t" + r1.nextGaussian());
System.out.println("-----");

Random r2 = new Random( seed: 50);
System.out.println("第二个种子为50的Random对象");
System.out.println("r2.nextBoolean():\t" + r2.nextBoolean());
System.out.println("r2.nextInt():\t\t" + r2.nextInt());
System.out.println("r2.nextDouble():\t" + r2.nextDouble());
System.out.println("r2.nextGaussian():\t" + r2.nextGaussian());
System.out.println("-----");
```

Run: TestSeed x

"C:\Program Files\Java\jdk-15\bin\java.exe"

第一个种子为50的Random对象

r1.nextBoolean(): true

r1.nextInt(): -1727040520

r1.nextDouble(): 0.6141579720626675

r1.nextGaussian(): 2.377650302287946

第二个种子为50的Random对象

r2.nextBoolean(): true

r2.nextInt(): -1727040520

r2.nextDouble(): 0.6141579720626675

r2.nextGaussian(): 2.377650302287946



相同“种子（seed）”的Random对象会生成相同的随机数。

生成“比较随机”的随机数的方法

通常使用以下方法生成“随机数”，它以当前时间为“种子”。

```
Random ran = new Random( System.currentTimeMillis() );
```

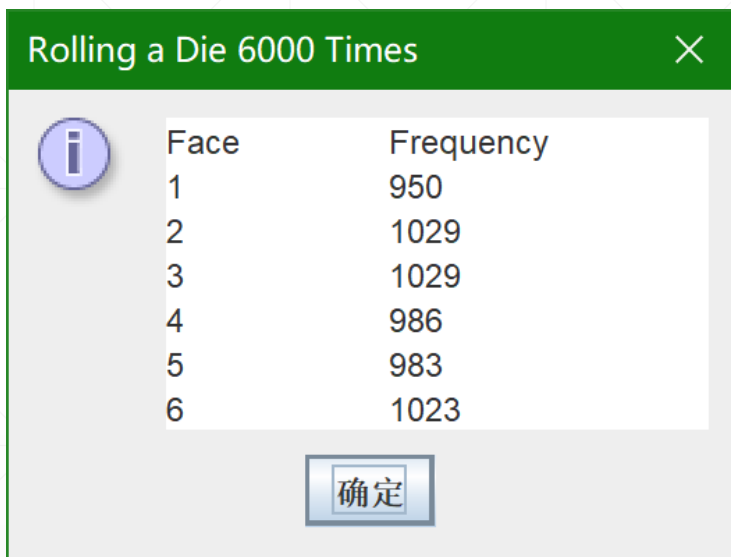
```
Random r4 = new Random(System.currentTimeMillis());  
System.out.println("以当前时间为种子的Random对象");  
System.out.println("r3.nextBoolean():\t" + r4.nextBoolean());  
System.out.println("r3.nextInt():\t\t" + r4.nextInt());  
System.out.println("r3.nextDouble():\t" + r4.nextDouble());  
System.out.println("r3.nextGaussian():\t" + r4.nextGaussian());
```

TestSeed.java

随机数的应用



示例：RollDie.java



Face	Frequency
1	950
2	1029
3	1029
4	986
5	983
6	1023

这个示例用随机数来模拟骰子滚动6000次的统计结果，请试着不看示例，自己完成这个功能。



巧妙地利用随机数，是很多游戏提升可玩性的重要手段。

完全“手写代码实现”随机数生成

一个可用于生成随机数的数学公式：

$$x_{n+1} = (ax_n + c) \bmod m$$

Multiplier

Increment

Modulus

Seed: x_0

纯随机数发生器公式

已知 $x_{n+1} = (ax_n + c) \bmod m$ 若令

$a = 2^{31}-1 = \text{int.MaxValue}$

$m = 7^5 = 16807$

$c = 0$

则当显示过 $2^{31}-2$ 个数之后，才可能重复。



编写一个方法，使用以上算法生成指定数目（比如1000个）的随机整数。

学习指导



在实际开发中，生成随机数是一个比较有用的功能，可以将实现它的相关代码整理并收集起来。



在学习过程中应该注意积累和整理各种有用的代码，分门别类地整理好，建立自己的代码仓库，这样一来，在编程时就能快速查找到相应的代码，从而有效地提升自己的开发效率。



使用自己收集整理资料库，通常比使用搜索引擎查找资料要高效得多，是一种高效的软件技术学习方法！

参数可变的方法

从 JDK5.0 起，Java支持可变参数的方法。

```
public class VariableArgumentsTest {  
    //定义一个支持可变参数的方法，此方法是获取所有输入参数中的最大值  
    //在数据类型后面加三个点，意味着这个方法能接收数目可变的参数  
    public static double max(double... values) {  
        double largest = Double.MIN_VALUE;  
        //传入的参数被当成是一个数组，可以使用foreach进行迭代  
        for (double v : values)  
            if (v > largest) largest = v;  
        return largest;  
    }  
    //main方法也可以改为“可变参数”  
    public static void main(String... args) {  
        //在四个传给max方法的实参数值中，最大的是300  
        //输出: Max:300.0  
        System.out.println("Max:" + max(...values: 1, 11, 300, 2, 3));  
    }  
}
```

可变参数方法的语法要求:

- 只能出现在方法参数列表的最后
- “...”位于变量类型和变量名之间，前后有无空格均可。
- 调用可变参数方法时，编译器为该可变参数隐含创建一个数组，在方法体中以数组的形式访问可变参数。



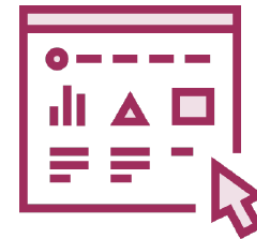
仔细观察.....

请看以下代码，你发现了有什么特殊之处吗？

```
public class MethodOverload {  
  
    public static void main(String[] args) {  
        System.out.println("The square of integer 7 is " + square(x: 7));  
        System.out.println("The square of double 7.5 is " + square(y: 7.5));  
    }  
  
    public static int square(int x) {  
        return x * x;  
    }  
  
    public static double square(double y) {  
        return y * y;  
    }  
}
```

MethodOverLoad.java

“方法重载”的要点



前述示例代码展示了Java的“**方法重载 (overload)**”特性。

满足以下条件的两个或多个方法构成“重载”关系：

1. 方法名相同；
2. 参数类型不同，参数个数不同，或者是参数类型的顺序不同。

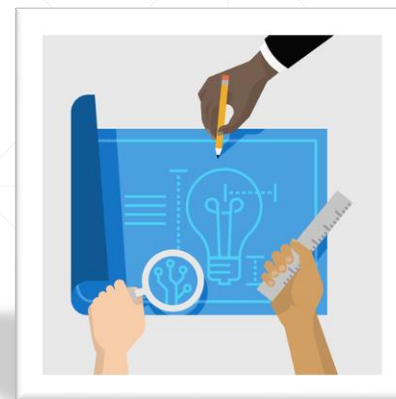


方法的返回值不作为方法重载的判断条件！



使用IntelliJ查看一下JDK中System.out.println()方法的源码，你发现了什么？

在了解了许多与方法相关的Java语法特性之后，
下面我们来谈谈在实际开发中，如何“用好方法”
的问题.....



在什么情况下应该定义“方法”？

当你发现某个功能在多个地方都需要时，那么将实现这个功能的代码提取出来，放到一个独立的方法中，并且给它起一个有意义的名字，就有必要了。

```
var sc = new Scanner(System.in);  
  
System.out.print("输入第一个数: ");  
double d1 = sc.nextDouble();  
  
System.out.print("输入第二个数: ");  
double d2 = sc.nextDouble();  
  
double result = d1 / d2;  
System.out.println("结果为 " + result);
```

这两段代码功能是完全一样的，仅仅只是输出的信息和接收输入的变量名不一样，是完全可以抽取出来作为一个独立的方法的。

参看：getInput.java

通过方法抽取消除重复代码

```
private static void addTwoNumberUseMethod() {  
    //调用getInput()方法读取用户输入  
    double d1 = getInput( prompt: "输入第一个数: ");  
    double d2 = getInput( prompt: "输入第二个数: ");  
  
    double result = d1 / d2;  
  
    System.out.println("结果为 " + result);  
}  
//将读取用户输入的功能抽取为单独的函数  
private static double getInput(String prompt) {  
    var sc = new Scanner(System.in);  
    System.out.print(prompt);  
    return sc.nextDouble();  
}
```

当需要修改数据输入方式，比如不是由用户现场输入，而是从文件读入时，可以只改这个函数，只要函数定义不变，内部不管怎么改，其他调用此函数的代码都无需修改。

另一种需要使用方法的场景——提升代码可读性

```
void doSomeJob(参数列表) {  
  
    // 校验所用到的原始数据的有效性  
    validateArguments(参数);  
  
    // 处理和加工数据  
    var result = process();  
  
    // 返回处理结果给外界  
    outputResult(result);  
}
```

左侧是某个方法的“骨架”，它又调用了另外三个方法完成特定的任务。

可以看到，当某方法要完成的工作比较多时，将整个过程划分为若干个步骤，为每个步骤编写特定的方法，会让整个代码变得很容易读懂。

最佳实践-1



在实际开发中，不要将所有代码都写到一个方法中，而应该将其拆分为多个“小”的方法，让每个方法“只干一件事”，方法的名字应该直接表明“它所干的这件事情是什么”。



单个方法的代码量要尽量精简在二三十行以内，最长不应该超过百行。太长的方法，应该将其拆成小的独立的方法，然后在“主控方法”中将它们“组合”起来。

最佳实践-2



在Java中，方法都是放在类中的，除了那些需要被外界直接调用的“主控方法”，其它所有的方法都应该设置为private的，不允许外界直接调用它们，这就是面向对象的“封装”原则的具体应用实例。



方法最好定义为“没有副作用”的，也就是说，只要传给它相同的数据，在任何情况下都返回相同的结果。



方法的名字不要怕长，可以将英文短句直接生成含义明确方法名，这样，可以省去书写注释的需求，比如：`deleteUserInfoFile`就是一个很好的方法名，一看这个名字就知道这个方法干什么事情。

下一讲，介绍一种特殊的方法调用——递归。